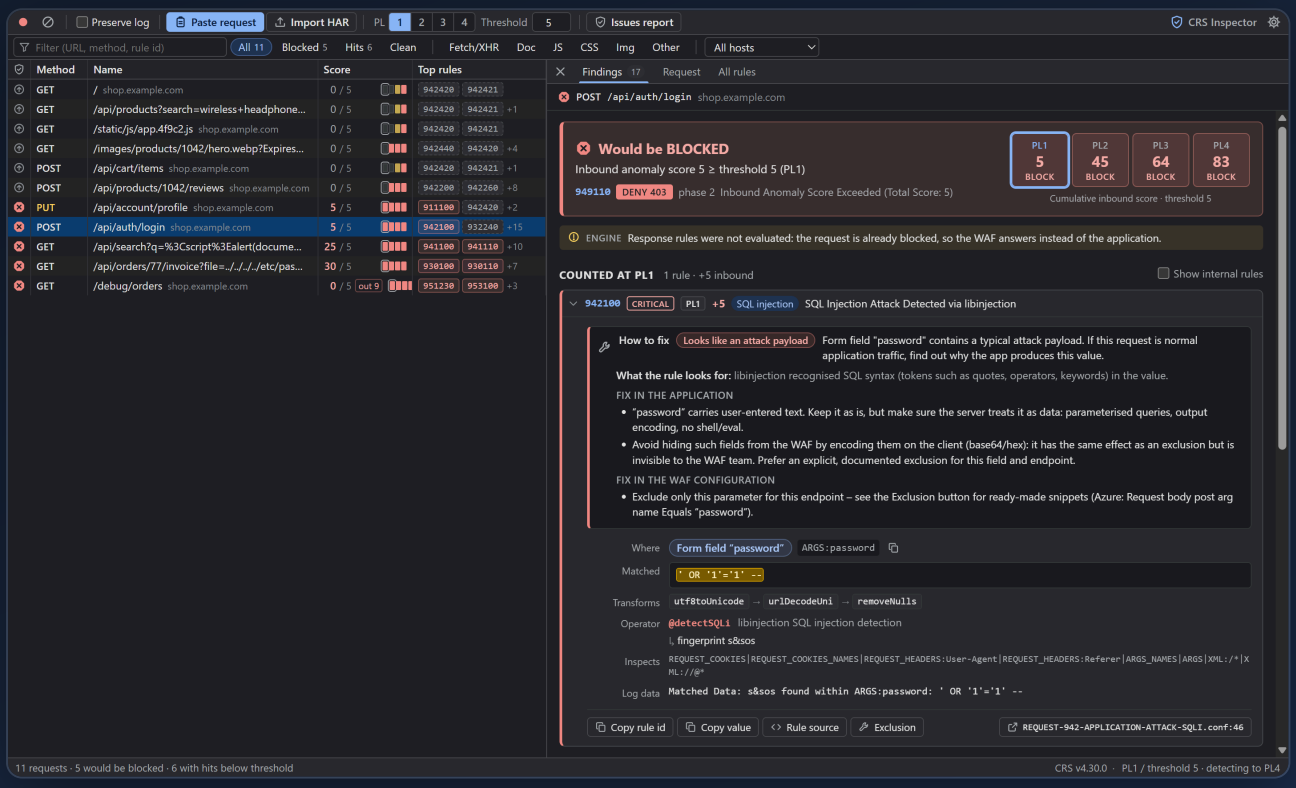
CHROME DEVTOOLS EXTENSION · OWASP CORE RULE SET

CRS Request Inspector

Installation & user guide — see which OWASP CRS / ModSecurity WAF rules your requests would trigger, where they match, and how to read paranoia levels and anomaly scores.



Bundled rule sets: CRS v4.30.0 · v4.25.2 (LTS) · v3.3.10

Everything runs locally – no request data leaves your browser

1 What the extension does

A Web Application Firewall (WAF) such as ModSecurity, Coraza or Azure Application Gateway / Front Door uses the OWASP Core Rule Set (CRS) to block attacks. CRS is generic – legitimate requests (signed URLs, rich text, JSON with special characters, unusual HTTP methods) can trigger it too. This extension runs the real CRS rules against your requests *before* they hit a WAF, so you see what would be blocked, which rule fires, on which part of the request, and why.

FEATURE	WHAT IT GIVES YOU
DevTools tab "OWASP CRS"	Next to *Network*. Every request of the inspected page is analysed automatically while DevTools is open.
Paste a request	Chrome "Copy as PowerShell / cURL / fetch", "Copy all as …", raw HTTP or HAR files – analysed instantly.
Verdict + paranoia matrix	Blocked / passes at your configured paranoia level, and the score at PL1–PL4 ("would start blocking at PL2").
Rule cards	Rule id with link to its source, the request part (JSON field, query arg, cookie, header, path), the matched text highlighted, decoding steps, the regex / libinjection fingerprint.
Exclusion helper	Copy-ready ModSecurity/CRS rule exclusions and an Azure WAF exclusion for false positives – test them immediately in the settings.

> **Accuracy.** The engine is a JavaScript re-implementation of ModSecurity v2 running the unmodified CRS rule files. It passes 99.9 % of the official CRS regression tests (5162/5166 for v4.30.0). Your production WAF may still differ in version, paranoia level, limits or exclusions – treat the result as "what a standard CRS installation would do".

Contents

1. What the extension does
2. Install the extension in Chrome
3. Open the OWASP CRS tab in DevTools
4. Reading the panel
5. Paranoia levels (PL1–PL4) and anomaly scores
6. Reading a rule card
7. Recommendations & issues report
8. Worked examples (demo shop)
9. Fixing false positives with exclusions
10. Why the extension and Azure WAF can disagree
11. Use your own WAF configuration (CRS / Azure)
12. Pasting requests
13. Settings
14. FAQ & troubleshooting

2 Install the extension in Chrome

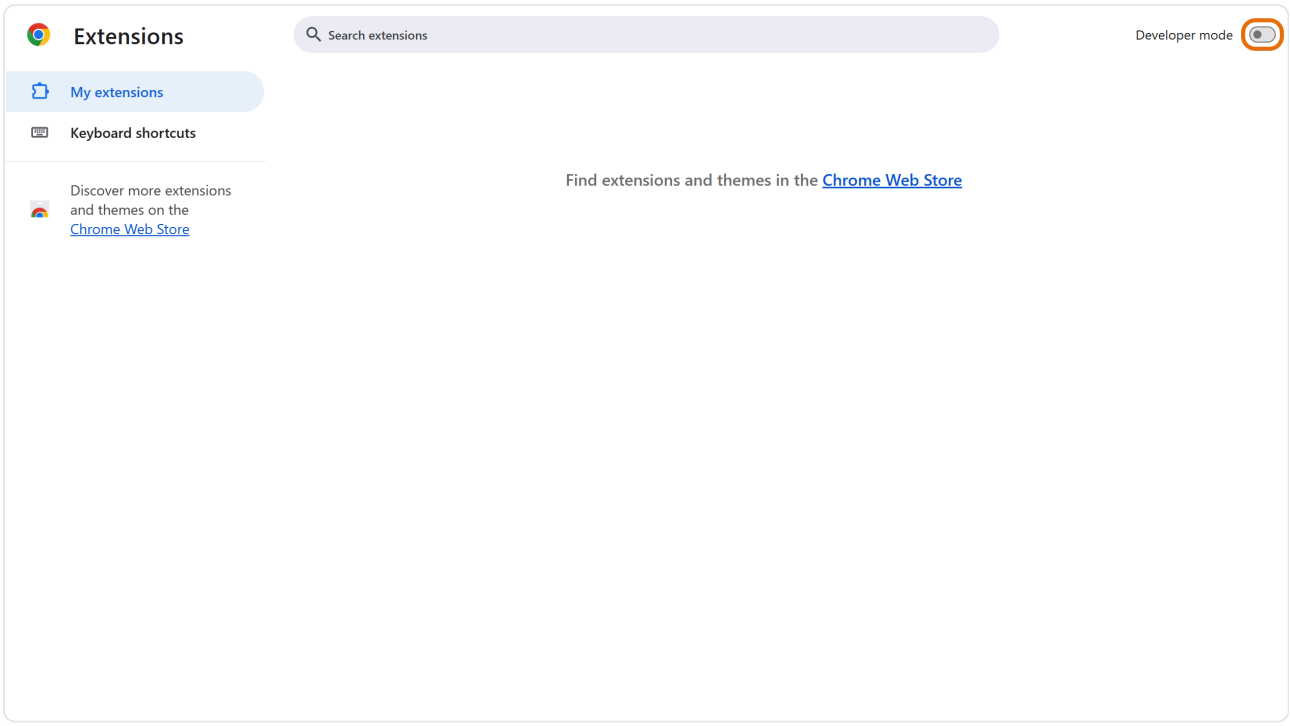
The extension is not in the Chrome Web Store; it is loaded “unpacked” from the project folder. This takes about a minute and only needs to be done once.

1 Open the extensions page

Type `chrome://extensions` into the address bar and press `Enter` (or menu ⋮ → **Extensions** → **Manage Extensions**).

2 Turn on Developer mode

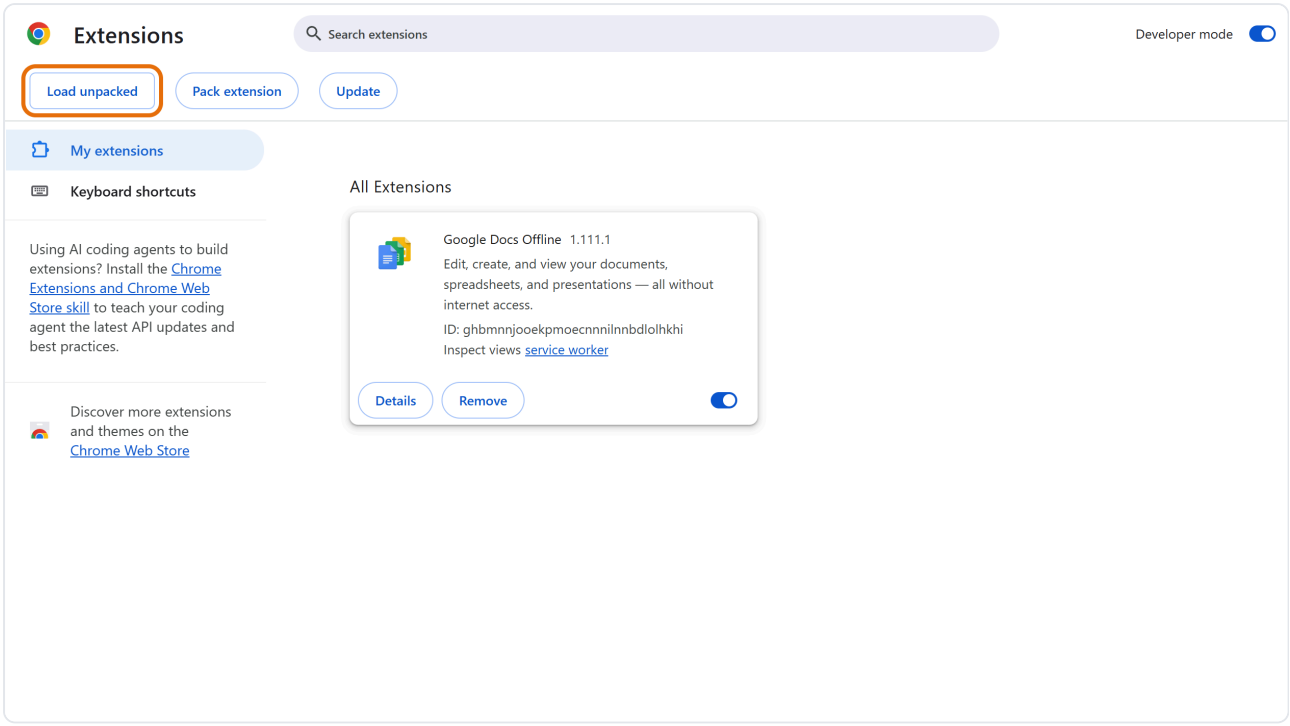
Switch on the **Developer mode** toggle in the top-right corner.



The Developer mode toggle (top right).

3 Click “Load unpacked”

Three new buttons appear. Click **Load unpacked**.



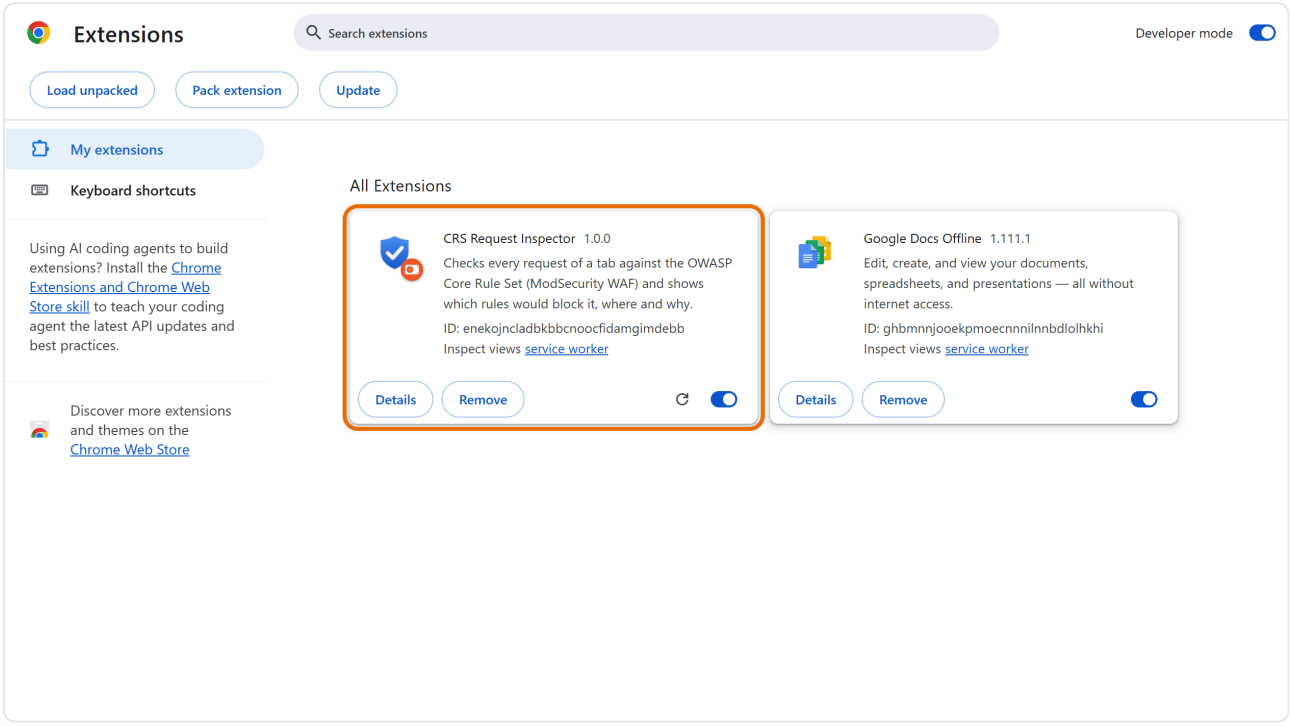
“Load unpacked” appears once Developer mode is on.

4 Select the `extension` folder

In the folder picker, open the project and select the folder named `extension` (the one that contains `manifest.json`), e.g. `C:\Users\<you>\src\crs-request-inspector\extension`, then click **Select Folder**. Do not select the project root or `extension\src`.

5 Check that it is installed

A card **CRS Request Inspector** appears. Make sure its toggle is on.



The installed extension. The ↻ icon reloads it after you update the files.

6 Optional: pin the toolbar icon

Click the puzzle icon 🧩 in the Chrome toolbar and pin **CRS Request Inspector**. Clicking the icon opens the analyser in a normal tab for pasting requests without DevTools.

> **Updating.** When the files change (e.g. new rule sets), click ↻ on the extension card, then close and reopen DevTools. Chrome may occasionally show a notice about developer-mode extensions at start-up – choose to keep the extension.

> **Permissions.** The extension only requests `storage` (for its settings). It reads requests through the DevTools API of the tab you are inspecting and never sends data anywhere.

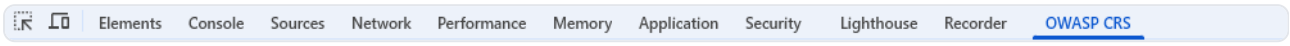
3 Open the OWASP CRS tab in DevTools

1 Open DevTools on your application

Press `F12` or `Ctrl`+`Shift`+`I` on the page you want to test.

2 Select the “OWASP CRS” tab

It sits in the DevTools tab bar next to *Network*. If the window is narrow, it is hidden behind the » overflow button.



The OWASP CRS tab in the DevTools tab bar (right of Recorder / AdBlock).

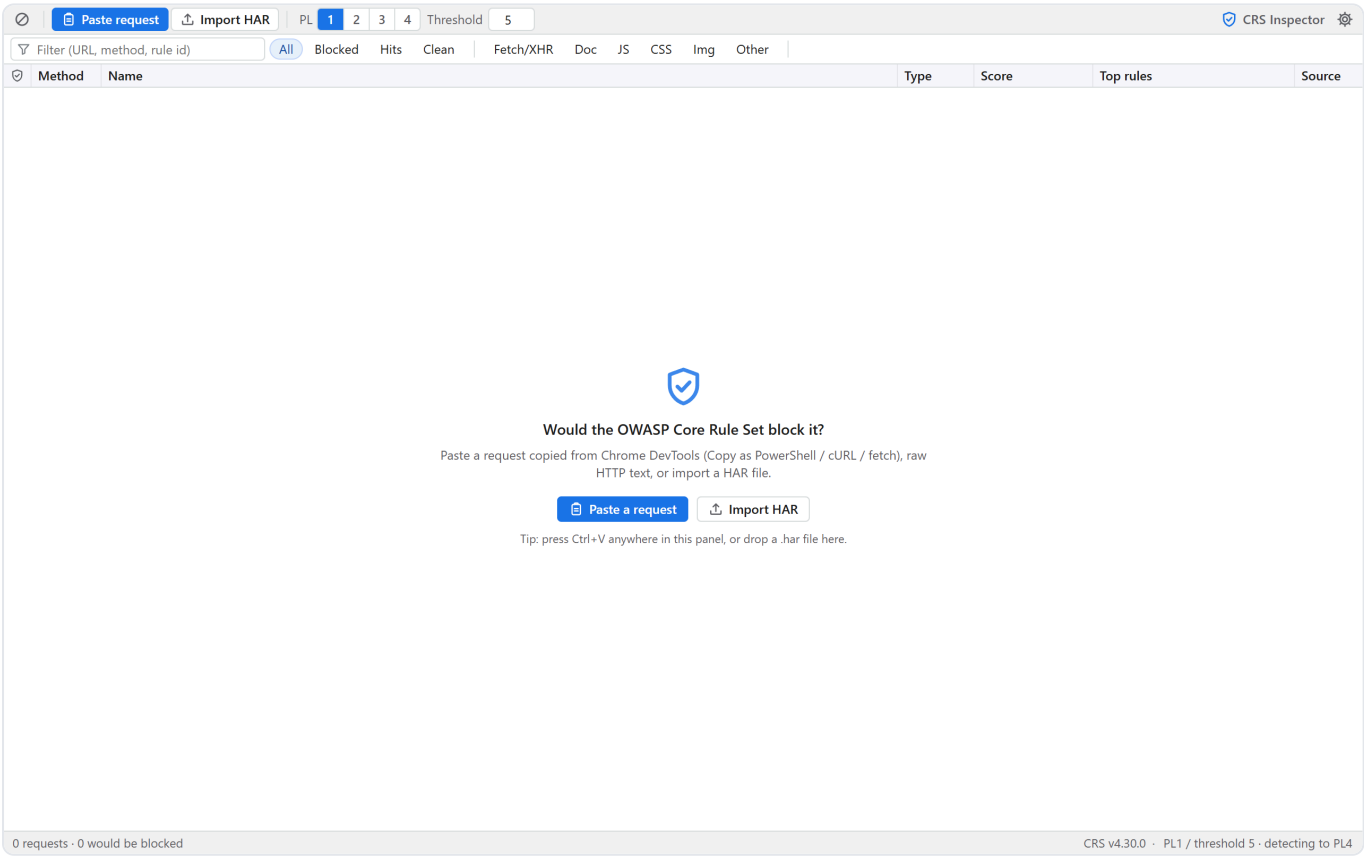
3 Reload the page

Requests are captured from the moment DevTools is open, like the Network tab. Press `F5` to capture the full page load, then use the application normally – every request appears in the list as it completes.

> **Nothing appears?** DevTools must be open *before* the requests are made – reload the page. The red ● record button in the toolbar must be on. Use **Preserve log** to keep entries across page navigations.

Without DevTools: the standalone page

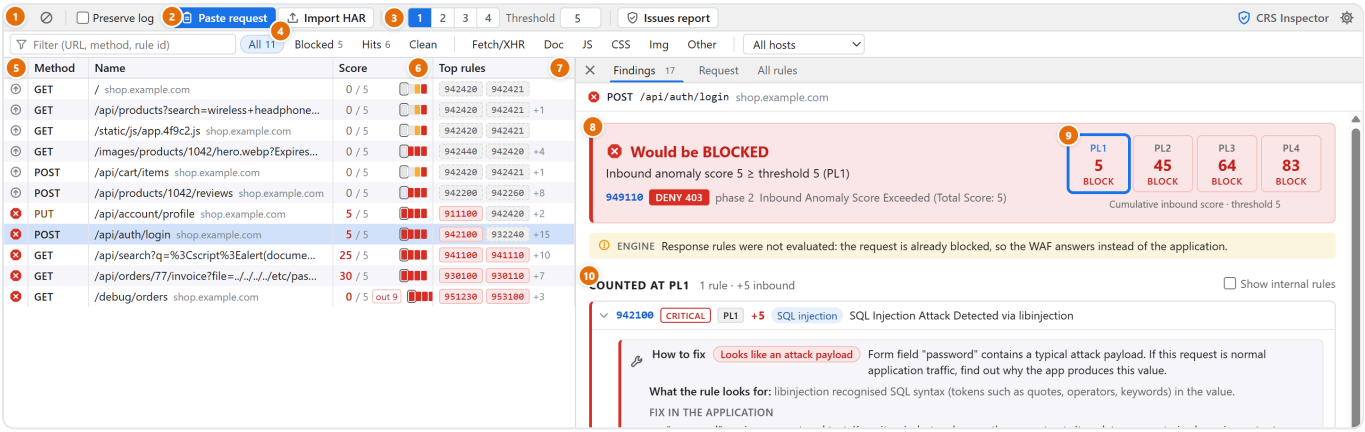
The toolbar icon opens the same interface in a normal tab (no live capture). Use it to paste requests or import HAR files.



Standalone page: paste a request or import a HAR file.

4 Reading the panel

Left: all requests (like the Network tab). Right: the analysis of the selected request.



1. **Record / clear / preserve log.** Stop or clear the capture; keep entries across navigations.
2. **Paste request / Import HAR.** Analyse requests copied from DevTools or a HAR file (also `Ctrl` + `V` anywhere in the panel, or drag & drop).
3. **Paranoia level & threshold.** The WAF configuration to simulate: blocking paranoia level PL1–PL4 and the inbound anomaly threshold (default 5). Changing them re-analyses everything. See section 5.
4. **Quick filters.** *All*, *Blocked* (would be blocked at the selected PL), *Hits* (rules matched but not blocked), *Clean*; resource types; host filter; free-text filter (URL, method or rule id).
5. **Status icon** per request – see the legend below.
6. **Score.** `8 / 5` = anomaly score at the selected PL / threshold. The four small boxes are PL1–PL4 (see below).
7. **Top rules.** Rule ids that matched. red = counts at the selected PL, grey = only at higher PLs; *+N* = more rules.
8. **Verdict.** *Would be BLOCKED*, *Passes* or *Clean*, with the reason and the rule that blocks (949110 = inbound anomaly score exceeded → HTTP 403).
9. **Paranoia matrix.** The cumulative score the request would reach at each PL and whether it would pass or be blocked. The outlined box is your selected PL.
10. **Findings.** One card per rule hit. *Counted at PLn* = rules that count towards the verdict; *Only at higher paranoia levels* = rules that would only count if you raise the level.

Status icons in the list

ICON	MEANING	WHAT TO DO
⊗ red	Would be **blocked** at the selected PL	Open it, look at the rule cards: real attack pattern, or a false positive to exclude?
⚠ orange	Rules count at the selected PL, but the total stays **below the threshold**	Passes today, but one more hit in the same request would block it.
⇧ grey	Rules match **only at higher PLs**	Passes at your PL. Relevant if you plan to raise the paranoia level.
✓ green	**Clean** – no CRS rule matched up to PL4	Nothing to do.

The four boxes in the Score column are PL1, PL2, PL3, PL4 from left to right: red = would be blocked at that level, orange = rules hit but the score stays below the threshold, empty = no hits. The box of the selected PL is outlined. Example: □■■■ means “passes at PL1, blocked from PL2 upwards”.

5 Paranoia levels (PL1–PL4) and anomaly scores

CRS does not block on a single "bad pattern". Every matching rule adds points to an **anomaly score**; the request is blocked when the score reaches the **threshold**. The **paranoia level** decides which rules take part.

What a paranoia level is

Every CRS rule belongs to one paranoia level (tag `paranoia-level/N`). A WAF configured with PL*n* runs **all rules from PL1 up to PL*n***. Higher levels add stricter rules: more attacks detected, but also many more false positives on normal traffic.

LEVEL	STRICTNESS	TYPICAL USE AND FALSE-POSITIVE RISK
PL1	Baseline (default)	Recommended starting point for almost every application. Few false positives; blocks clear attacks (SQL injection, XSS, path traversal, …). Most managed WAF setups run at this level.
PL2	Elevated	Applications with sensitive data. Adds more SQL/XSS/RCE patterns and checks on special characters. False positives are common with free text, rich text, tokens and signed URLs – needs tuning.
PL3	High	Banking / high-security. Counts special characters, very strict protocol checks. Many false positives; requires a dedicated tuning effort.
PL4	Paranoid	Extremely strict (e.g. allows only a very small character set in arguments). Nearly every real application needs extensive exclusions.

How the score works

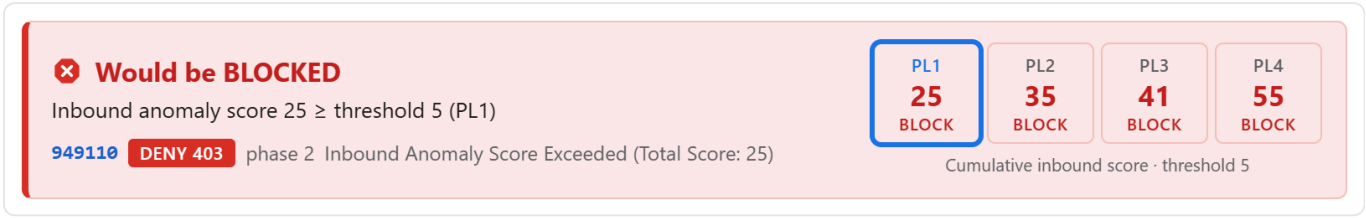
RULE SEVERITY	POINTS	EFFECT WITH THE DEFAULT THRESHOLD 5
CRITICAL	+5	One hit is enough to block.
ERROR	+4	Blocks together with any other hit.
WARNING	+3	Needs at least one more hit.
NOTICE	+2	Needs further hits.

A rule that matches in several places (e.g. two query parameters) adds its points for each match. At the end of the request phase, rule **949110** compares the total with the threshold and denies the request with **HTTP 403** (shown as `949110 DENY 403` in the verdict). Responses are scored the same way with the outbound threshold (default 4, rule 959100).

Reading the PL matrix



Passes at PL1, blocked from PL2. No rule counts at PL1 (score 0 < 5). If the WAF ran PL2, rule 942440 (+5) would block it; at PL3 the score is 8, at PL4 19.



Blocked at every level. A SQL-injection payload scores 8 already at PL1 (≥ 5) – rule 949110 would answer with 403.



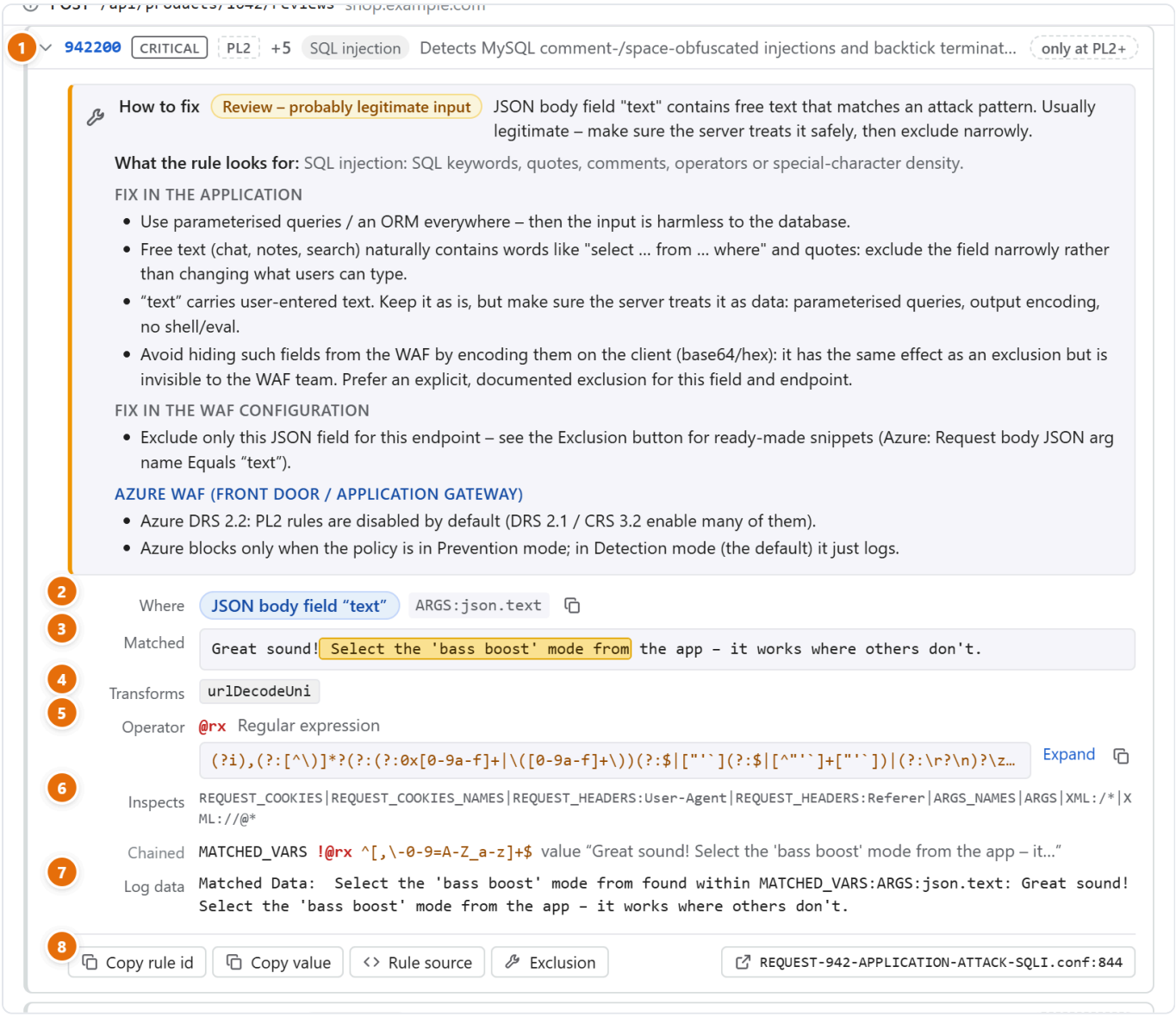
Clean. No rule matched up to PL4.

The numbers are cumulative: the PL3 box is the sum of all hits from PL1, PL2 and PL3 – exactly what a WAF configured with PL3 would compute. The toolbar PL buttons choose the *blocking* level you want to simulate; the extension always also evaluates up to PL4 (*detection* level, see Settings) so you can see what would happen if you raised it.

Which PL does our WAF use? Ask whoever manages the WAF policy (or check its configuration: `tx.blocking_paranoia_level` in CRS 4, `tx.paranoia_level` in CRS 3, the managed rule set and disabled rules in Azure). If unknown, assume **PL1** and threshold **5**.

6 Reading a rule card

Click a rule in the findings list to expand it. Every card starts with a **How to fix** box (section 7), followed by the technical details. Example: rule 942440 matching a signed image URL.



1 **Rule header.** Rule id (links to the exact line in the CRS source on GitHub), severity, paranoia level, points added (*+5*), attack category and the rule's message. *only at PL2+* = does not count at the selected PL.

✓ **How to fix** – assessment (likely false positive / review / attack / policy), what the rule looks for, and concrete changes for the application, the WAF and Azure WAF – see section 7.

2 **Where** – the part of the request that matched, in plain words (*Query arg "Signature"*, *JSON body field "content"*, *Cookie "session"*, *Header "User-Agent"*, *URL path*) and the ModSecurity variable name (`ARGS:Signature`) you need for exclusions.

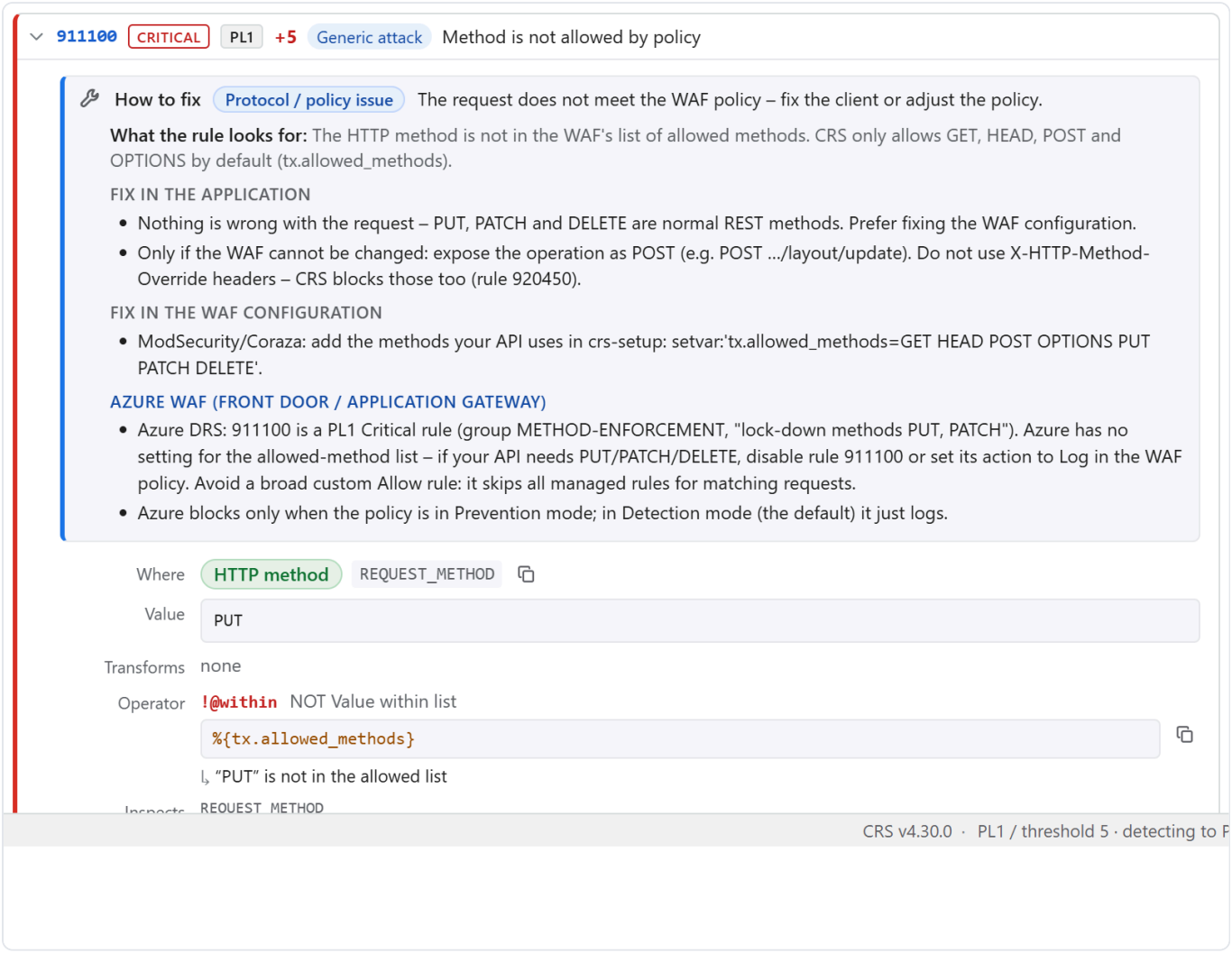
3 **Matched** – the value, with the part that triggered the rule highlighted. The value is shown after decoding (see 4); expand *Original value* to see it as sent.

4. **Transforms** – how the WAF normalised the value before matching (URL-decoding, HTML entity decoding, lower-casing, comment removal …). Attackers use encodings to hide payloads; the WAF decodes first.
5. **Operator** – the test that fired: `@rx` regular expression (copyable), `@detectSQLi` / `@detectXSS` with the libinjection fingerprint, `@pm` keyword list, etc.
6. **Inspects** – all request parts this rule looks at. **Chained** – extra conditions that must also be true (here: the value must *not* look like a JWT).
7. **Log data** – exactly what the WAF would write to its audit log (useful to match against WAF logs).
8. **Actions** – copy rule id / value, show the rule's SecLang source, open the **Exclusion** helper (section 9), open the rule file on GitHub.

> **Notes above the findings** explain how the request was interpreted – e.g. PARSER "Detected HTTP/2 pseudo-headers …" (Chrome sends HTTP/2; the `:authority`, `:path` … headers are not real headers) or a body that could not be parsed as JSON (ModSecurity rejects such requests with rule 200002).

7 Recommendations & issues report

For every finding the extension explains what to change – so developers know whether to fix the website or ask for a WAF exclusion.

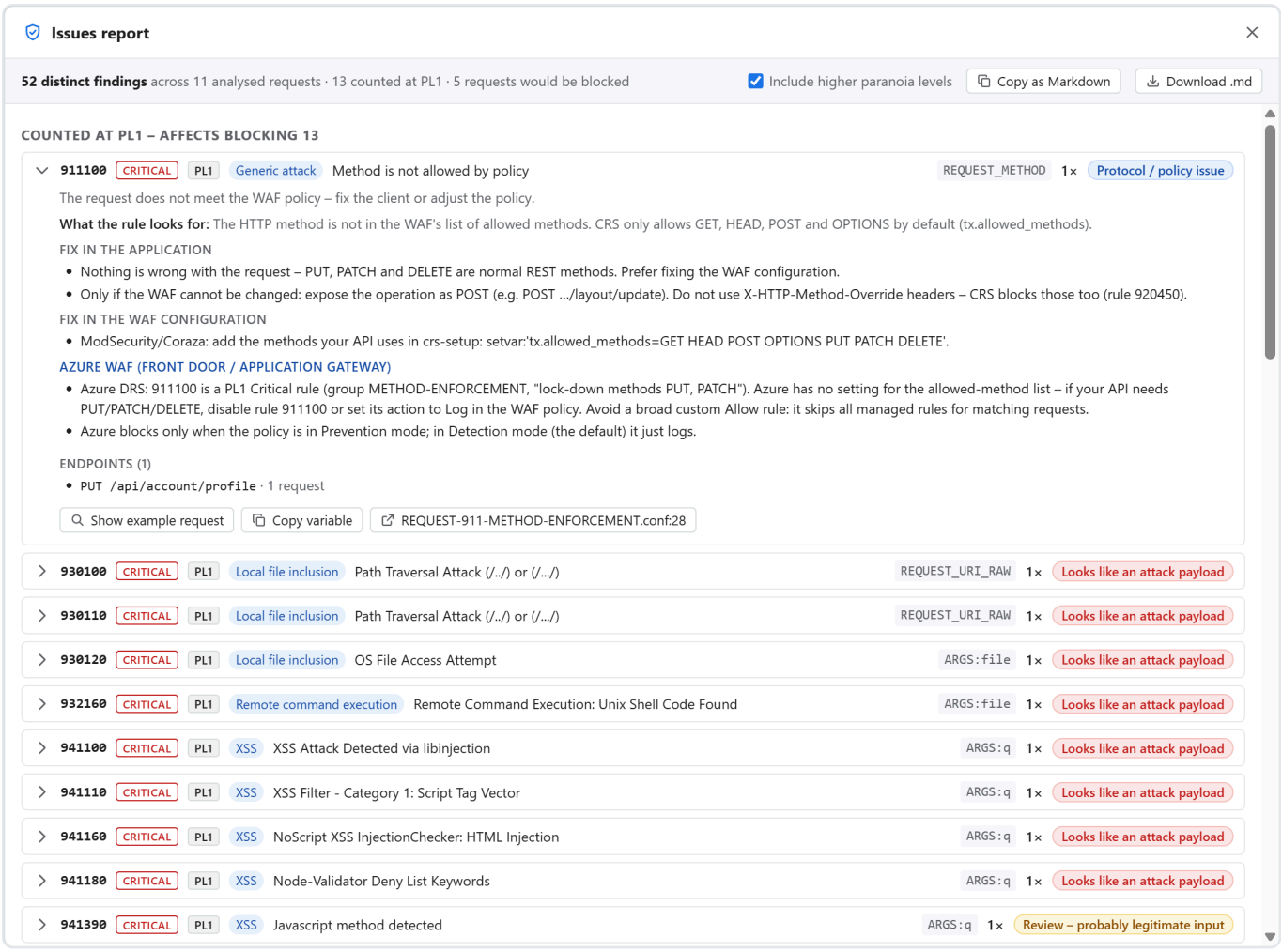


"How to fix" for a DELETE request blocked by 911100 (method not allowed).

ASSESSMENT	MEANING AND TYPICAL ACTION
Likely false positive	The matched value is random data – a JWT, signature, base64 token, hash or UUID. It only looks like attack syntax. **Action:** WAF exclusion for that variable (or, if your code generates it, a format without special characters).
Review – probably legitimate input	User-entered text or HTML (reviews, comments, search). Usually legitimate. **Action:** confirm the server handles it safely (parameterised queries, output encoding, no shell/eval), then exclude only that field on that endpoint.
Looks like an attack payload	A strong PL1 signature (libinjection SQLi/XSS, path traversal). **Action:** if this was normal application traffic, find out why the app produces such a value – otherwise the WAF is doing its job.
Protocol / policy issue	The request breaks a policy (method, content type, missing header, size limit). **Action:** fix the client, or adjust the WAF policy (e.g. allowed methods).
Information leak	The response shows error details or stack traces. **Action:** disable detailed errors in production.

Issues report – all findings in one list

Click **Issues report** in the toolbar. All findings of the captured requests are grouped by *rule + request part* with the number of requests, the affected endpoints (ids generalised, e.g. `/orders/:id`) and the recommendation. Findings that count at your paranoia level come first. **Copy as Markdown** / **Download .md** produce a report you can paste into a ticket or pull request.



Issues report with one finding expanded.

8 Worked examples (demo shop)

These examples come from the fictitious **Example Shop** in `demo/` (`node demo/server.mjs`, then open DevTools → **OWASP CRS** and click *Browse normally* / *Send attack samples*). They are typical of what the panel finds on real applications.

Signed image URLs

Product images are loaded through a CDN with signed URLs, e.g. `/images/products/1042/hero.webp?Expires=…&Signature=…&Key-Pair-Id=…`.

PL	SCORE	WHY
PL1	0 · pass	No PL1 rule matches – at the default level these requests are fine.
PL2	5 · block	The `Signature` value is URL-safe base64 and contains `--` and `~`. **942440 "SQL Comment Sequence Detected"** sees `--` (an SQL comment) → +5 = blocked.
PL3–4	11–28 · block	942431/942432 count special characters, 920273 rejects characters outside a strict set.

Conclusion: at PL1 nothing is blocked. If the WAF runs at PL2 or higher, **every** product image would get a 403 – a classic false positive. The signature is random data, not user input, so it is safe to exclude `ARGS:Signature` from the SQL-injection rules for the `/images/` path.

Product reviews (free text)

`POST /api/products/1042/reviews` sends `{"rating":5,"text":"Great sound! Select the 'bass boost' mode from the app – it works where others don't."}`. At PL1 it passes; at PL2 rules such as **942200** see SQL-like words and quotes ("Select … from", `'`) and the score reaches 15. The text is legitimate, but it *is* user input: keep it parameterised/escaped in the application and, if you run PL2, exclude only `ARGS:json.text` on the reviews endpoint.

Session cookie containing a JWT

Almost every request shows hits on **942420** (PL3) and **942421** (PL4) – "Restricted SQL character anomaly detection (cookies)". The cause is the `shop_session` cookie: it holds a full JWT whose signature contains many `-` characters, which these rules count as SQL special characters.

- **Impact:** none at PL1/PL2 – and Azure WAF does not run PL3/PL4 rules at all.
- **Application option:** store an opaque random session id in the cookie and keep the JWT server-side – smaller cookies, no special characters, token not exposed to the browser.
- **WAF option (only if you run CRS at PL3+):** exclude `REQUEST_COOKIES:shop_session` from 942420/942421.

PUT / PATCH / DELETE API calls

`PUT /api/account/profile` scores 5 at PL1 because of **911100 "Method is not allowed by policy"**: standard CRS allows only GET, HEAD, POST and OPTIONS. This is a WAF policy question, not a bug in the request – see section 10 for why it may not be blocked in your environment.

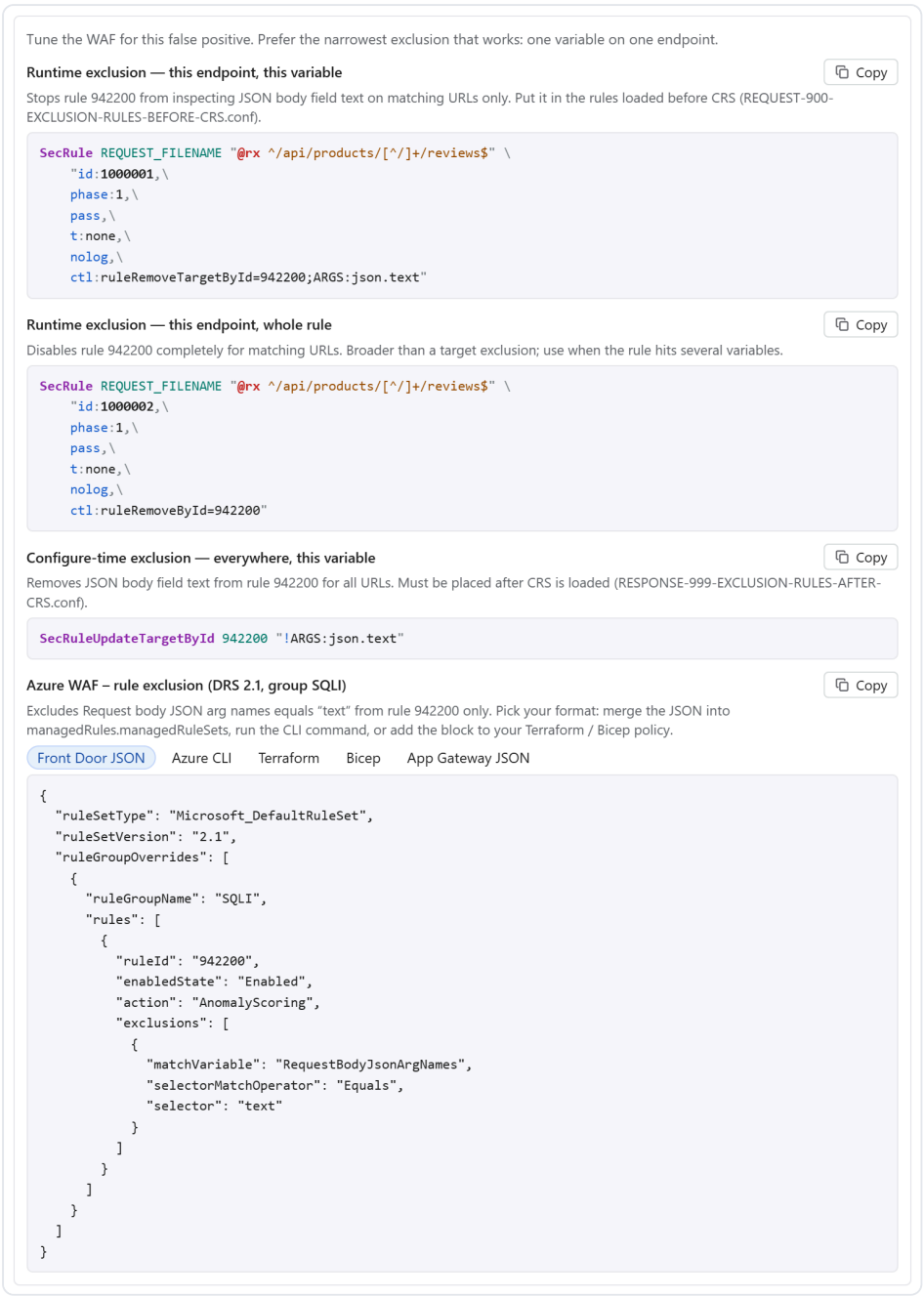
Client-side encoded fields

Some applications encode free text on the client (base64, hex) before sending it. Encoded content passes the WAF because the WAF cannot read it – the same effect as an exclusion, but invisible to whoever operates the WAF. That is acceptable only if the server validates and safely handles the decoded text; an explicit, documented exclusion for that one field on that one endpoint is the more transparent alternative.

> **Real attack or false positive?** Ask: does the matched value come from a user, and could it reach a database, a shell or an HTML page? Random tokens, signatures, hashes and encrypted blobs → false positive. Free text from a form, search box or review → real risk; exclude only narrowly (one field, one endpoint) and make sure the application escapes/parameterises it.

9 Fixing false positives with exclusions

Click **Exclusion** on a rule card. The helper generates snippets for the matched variable and endpoint – ids, numbers and UUIDs in the path are generalised (`[^/]+` , `[0-9]+`).



Tune the WAF for this false positive. Prefer the narrowest exclusion that works: one variable on one endpoint.

Runtime exclusion — this endpoint, this variable Copy

Stops rule 942200 from inspecting JSON body field text on matching URLs only. Put it in the rules loaded before CRS (REQUEST-900-EXCLUSION-RULES-BEFORE-CRS.conf).

```
SecRule REQUEST_FILENAME "@rx ^/api/products/[^/]+/reviews$" \
    "id:1000001,\
    phase:1,\
    pass,\
    t:none,\
    nolog,\
    ctl:ruleRemoveTargetById=942200;ARGS:json.text"
```

Runtime exclusion — this endpoint, whole rule Copy

Disables rule 942200 completely for matching URLs. Broader than a target exclusion; use when the rule hits several variables.

```
SecRule REQUEST_FILENAME "@rx ^/api/products/[^/]+/reviews$" \
    "id:1000002,\
    phase:1,\
    pass,\
    t:none,\
    nolog,\
    ctl:ruleRemoveById=942200"
```

Configure-time exclusion — everywhere, this variable Copy

Removes JSON body field text from rule 942200 for all URLs. Must be placed after CRS is loaded (RESPONSE-999-EXCLUSION-RULES-AFTER-CRS.conf).

```
SecRuleUpdateTargetById 942200 "!ARGS:json.text"
```

Azure WAF – rule exclusion (DRS 2.1, group SQLI) Copy

Excludes Request body JSON arg names equals “text” from rule 942200 only. Pick your format: merge the JSON into managedRules.managedRuleSets, run the CLI command, or add the block to your Terraform / Bicep policy.

Front Door JSON | Azure CLI | Terraform | Bicep | App Gateway JSON

```
{
  "ruleSetType": "Microsoft_DefaultRuleSet",
  "ruleSetVersion": "2.1",
  "ruleGroupOverrides": [
    {
      "ruleGroupName": "SQLI",
      "rules": [
        {
          "ruleId": "942200",
          "enabledState": "Enabled",
          "action": "AnomalyScoring",
          "exclusions": [
            {
              "matchVariable": "RequestBodyJsonArgNames",
              "selectorMatchOperator": "Equals",
              "selector": "text"
            }
          ]
        }
      ]
    }
  ]
}
```

SNIPPET	WHEN TO USE
Runtime exclusion — this endpoint, this variable	**Preferred.** Removes one variable from one rule on matching URLs only. Goes into the rules loaded *before* CRS.
Runtime exclusion — this endpoint, whole rule	When the rule hits many variables on that endpoint.
Configure-time — everywhere, this variable	The variable is never dangerous anywhere (e.g. a CSRF token). Goes *after* CRS.
Azure WAF	Exclusion JSON for Application Gateway / Front Door WAF policies (match variable, selector, rule).

Verify an exclusion before rolling it out

1. **Copy the snippet**

   Use the **Copy** button next to it.

2. **Paste it into the settings**

   Open settings (⚙), paste it into *Custom rules before CRS* (runtime exclusions) or *after CRS* (`SecRuleUpdateTargetById`), click **Apply**.

3. **Check the result**

   All requests are re-analysed. The rule should disappear from the card list and the verdict should change to *Passes*. Then hand the snippet to the WAF owner.

10 Why the extension and Azure WAF can disagree

With the default profile the extension shows what a **standard CRS installation in blocking mode** would do – load your own policy (section 11) to simulate your actual Azure WAF. If a request it marks as blocked goes through in your environment, the WAF in front of the application is configured differently – or is not enforcing at all.

Which WAF is in front of the site?

Look at the response headers in the Network tab: `x-azure-ref` means the site is served through **Azure Front Door**; Application Gateway WAF usually leaves no header but answers blocked requests with its own 403 page. Azure WAF policies use Microsoft's Default Rule Set (DRS 2.x – based on CRS 3.3 plus Microsoft rules), not the open-source CRS 4 that the extension uses by default – switch the profile to **Azure WAF** and import your policy (section 11) for an exact simulation.

POSSIBLE REASON	EXPLANATION
Policy in Detection mode	Azure enables managed rules **in Detection mode by default**: matches are only logged, nothing is blocked. Very common in dev/test environments.
Front Door Standard tier	Managed rule sets (DRS) are only available on **Front Door Premium** (and classic). On Standard only custom rules exist – no CRS-style detection at all.
No WAF policy on this domain	The WAF policy must be attached to the endpoint / domain through a Front Door security policy.
Rule disabled or excluded	911100 (or another rule) may be disabled, set to Log, covered by an exclusion or by a custom Allow rule.
Different rule set	DRS differs from CRS 4: no PL3/PL4 rules, PL2 rules disabled by default in DRS 2.2, some rules replaced by Microsoft rules (e.g. 942440 → 99031002). The *How to fix* box shows these Azure notes per rule.

How to verify

1 Check the WAF policy

Azure portal → Front Door profile → *Security policies* → WAF policy → *Policy settings*: **Mode** (Detection / Prevention); *Managed rules*: rule set version and the state of rule 911100. With the Azure CLI:

```
az afd profile show -g <resource-group> --profile-name <profile> --query sku.name
az afd security-policy list -g <resource-group> --profile-name <profile> -o table
az network front-door waf-policy show -g <resource-group> -n <policy> --query "
{mode:policySettings.mode, state:policySettings.enabledState, rules:managedRules.managedRuleSets[].
{type:ruleSetType, version:ruleSetVersion}}"
```

2 Look at the WAF log

If diagnostic logs are sent to Log Analytics, this shows what the WAF decided for the PUT request (`action_s` = Log / Block / AnomalyScoring):

```
AzureDiagnostics
| where Category == "FrontDoorWebApplicationFirewallLog"
| where requestUri_s has "/api/account/profile"
| project TimeGenerated, ruleName_s, action_s, policyMode_s, details_msg_s, requestUri_s
| order by TimeGenerated desc
```

3 Optional live test

An obvious, harmless test string shows immediately whether the WAF blocks: a **403** from Front Door means Prevention mode with managed rules is active; a normal response (e.g. **401** from the App Service login) means the WAF does not block. Inform your security team first – this deliberately triggers a WAF alert.

```
curl -s -o /dev/null -w "%{http_code}\n" "https://shop.example.com/?crs-
test=%3Cscript%3Ealert(1)%3C%2Fscript%3E"
```

11 Use your own WAF configuration

The extension supports two WAF profiles: **OWASP CRS** (ModSecurity, Coraza and other CRS-based WAFs) and **Azure WAF policy** (Front Door / Application Gateway). Both can import your real configuration.

PROFILE	IMPORT FORMATS	WHAT IS APPLIED
OWASP CRS ModSecurity, Coraza, CRS-based WAFs	`crs-setup.conf`, REQUEST-900 / RESPONSE-999 exclusion files, any SecRule/SecAction files (Settings → *Import ModSecurity / Coraza / CRS configuration*)	Paranoia levels, thresholds, CRS version (from `tx.*` setvars) become settings; exclusions and other rules go into the custom rules before / after CRS. Warns about `SecRuleEngine DetectionOnly` and `Include` lines.
Azure WAF policy Front Door, Application Gateway	Policy JSON (portal JSON View, `az … waf-policy show`, ARM export), **Terraform** (`azurerm_cdn_frontdoor_firewall_policy`, `azurerm_frontdoor_firewall_policy`, `azurerm_web_application_firewall_policy`), **Bicep** – detected automatically	Custom rules, managed rule set version, rule overrides, exclusions, rule-set action, mode (see below).

AWS WAF and Cloudflare configurations are recognised but cannot be simulated – they use their own rule engines. Exclusion snippets are offered for every profile: ModSecurity SecLang, and for Azure as Front Door JSON, Azure CLI, Terraform, Bicep and Application Gateway JSON.

Azure WAF policy

1. **Export the policy**

   Azure portal → your WAF policy → *Overview* → **JSON View** → copy, use your Terraform / Bicep file, or the Azure CLI:

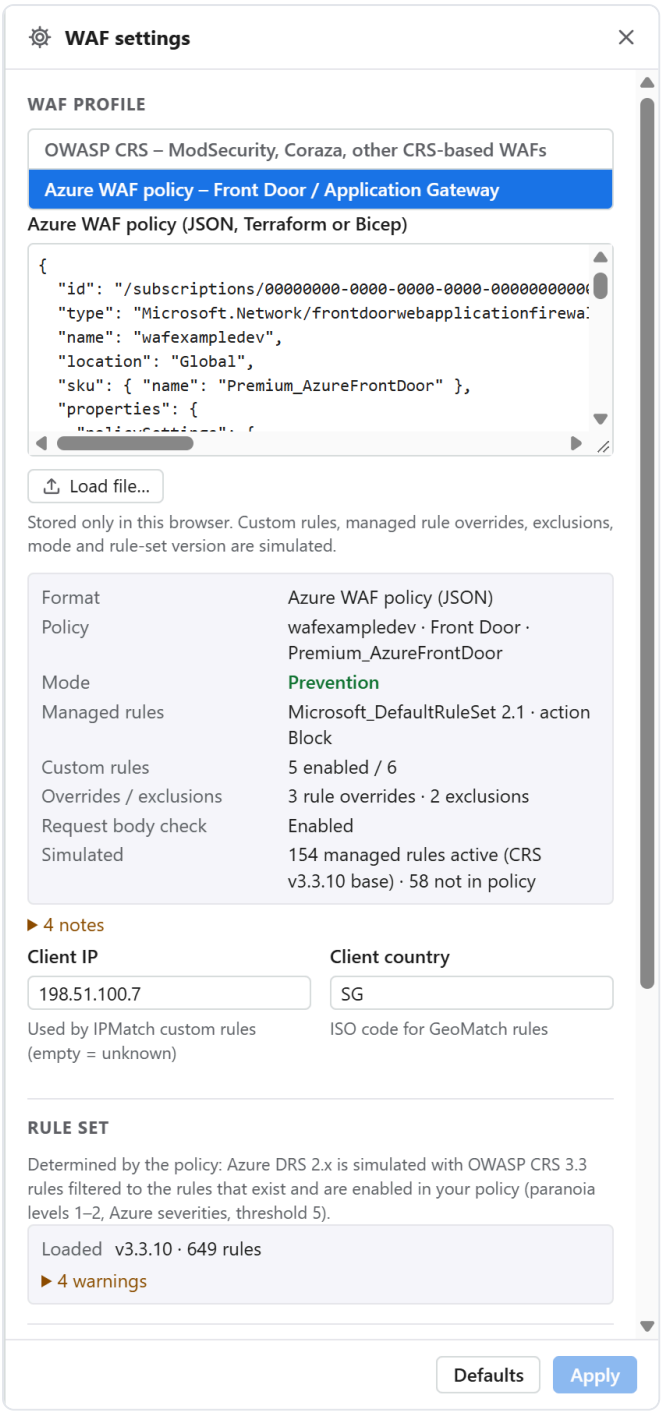
   ```
   az network front-door waf-policy show -g <resource-group> -n <policy-name> > waf-policy.json
   ```

   ARM/Bicep exports and Application Gateway policies (`az network application-gateway waf-policy show`) work too.

2. **Load it into the extension**

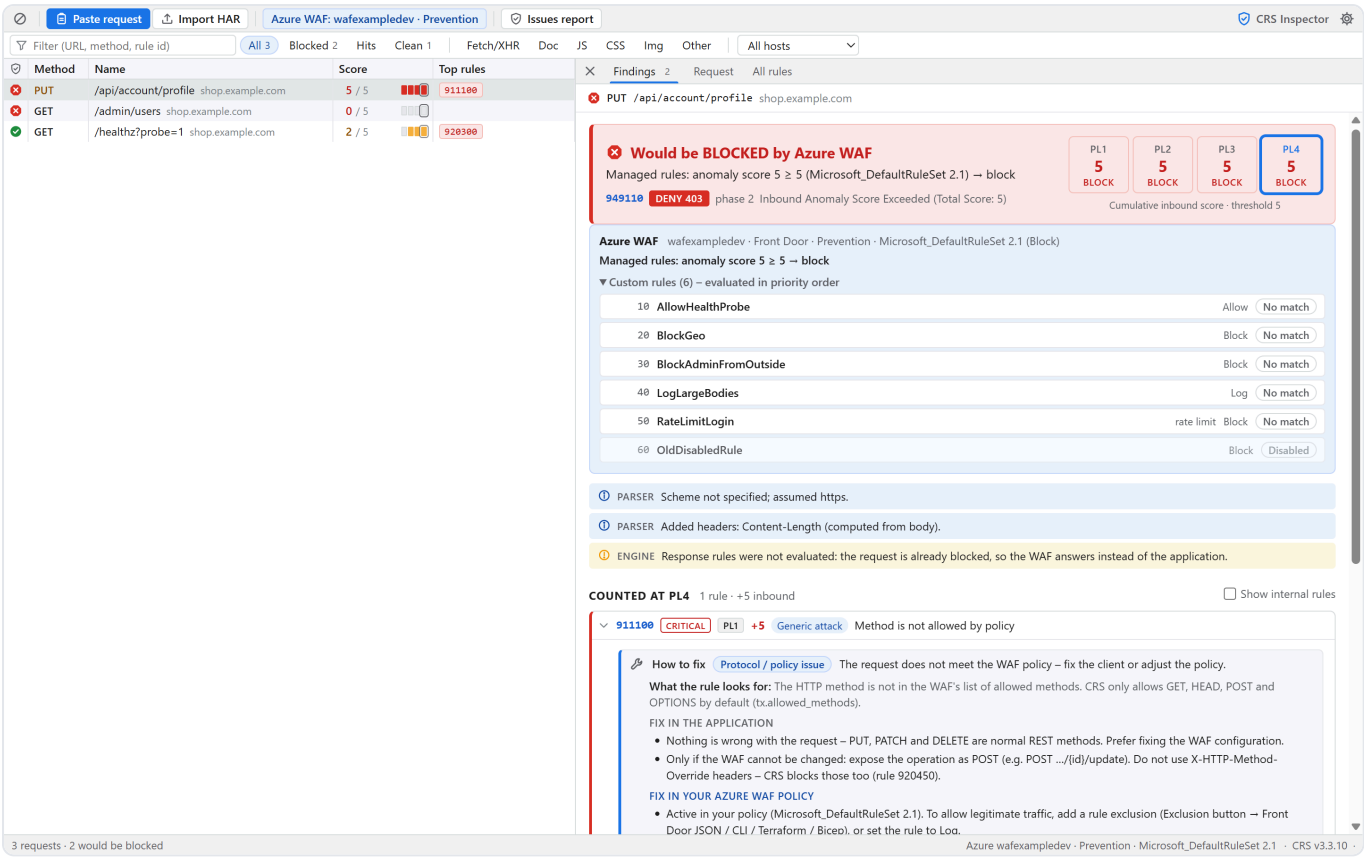
   Settings (⚙) → **WAF profile** → *Azure WAF policy* → paste it or **Load file…** (.json, .tf, .bicep) – the format is detected automatically. A summary appears immediately (mode, rule set, custom rules, overrides, exclusions) together with notes about anything that cannot be simulated. Optionally set the **client IP** and **client country** used by IPMatch/GeoMatch custom rules. Click **Apply**.

3. **Read the Azure verdictEvery request now shows *Would be BLOCKED by Azure WAF*, *Allowed by Azure WAF* or *Passes*, which component decided (a custom rule with its priority, or the managed rule set's anomaly score), and an Azure WAF panel listing each custom rule in priority order with the result of every match condition and the value it saw.**



Imported policy summary in the settings.

SIMULATED	HOW
Custom rules	**Priority order; Allow / Block / Redirect stop evaluation, Log continues; all conditions AND, values OR; transforms; negation; size operators compare byte length; IPMatch / GeoMatch via the client IP / country setting.**
Managed rules (DRS)	**Only rules that exist in your DRS version (2.2, 2.1, 2.0, 1.x) and are enabled in the policy; Azure's severities; anomaly threshold 5; rule-set action (Block / Log / Redirect).**
Rule overrides	**Disabled rules are removed; rule action *Log* reports the rule without adding to the score; *Block* / *Redirect* block immediately; *Allow* stops evaluation.**
Exclusions	**Header / cookie / query / post / JSON argument exclusions at policy, rule-set, group and rule level (Equals, StartsWith, EndsWith, Contains, EqualsAny).**
Mode	**Detection → "logged only"; disabled policy → nothing inspected.**



PUT request blocked by the managed rules (911100) of DRS 2.1 in Prevention mode; the Azure WAF panel shows that no custom rule ended the evaluation.

> **Not simulated:** Microsoft Threat Intelligence rules (rule ids 99xxxxxx – not public), the Bot Manager rule set (IP reputation), rate limits (a single request cannot exceed a rate), ASN / client-fingerprint / service-tag conditions. DRS rules are approximated with the equivalent OWASP CRS 3.3 rules they are based on – Microsoft may have tuned individual signatures. The policy JSON is stored only in your browser.

12 Pasting requests

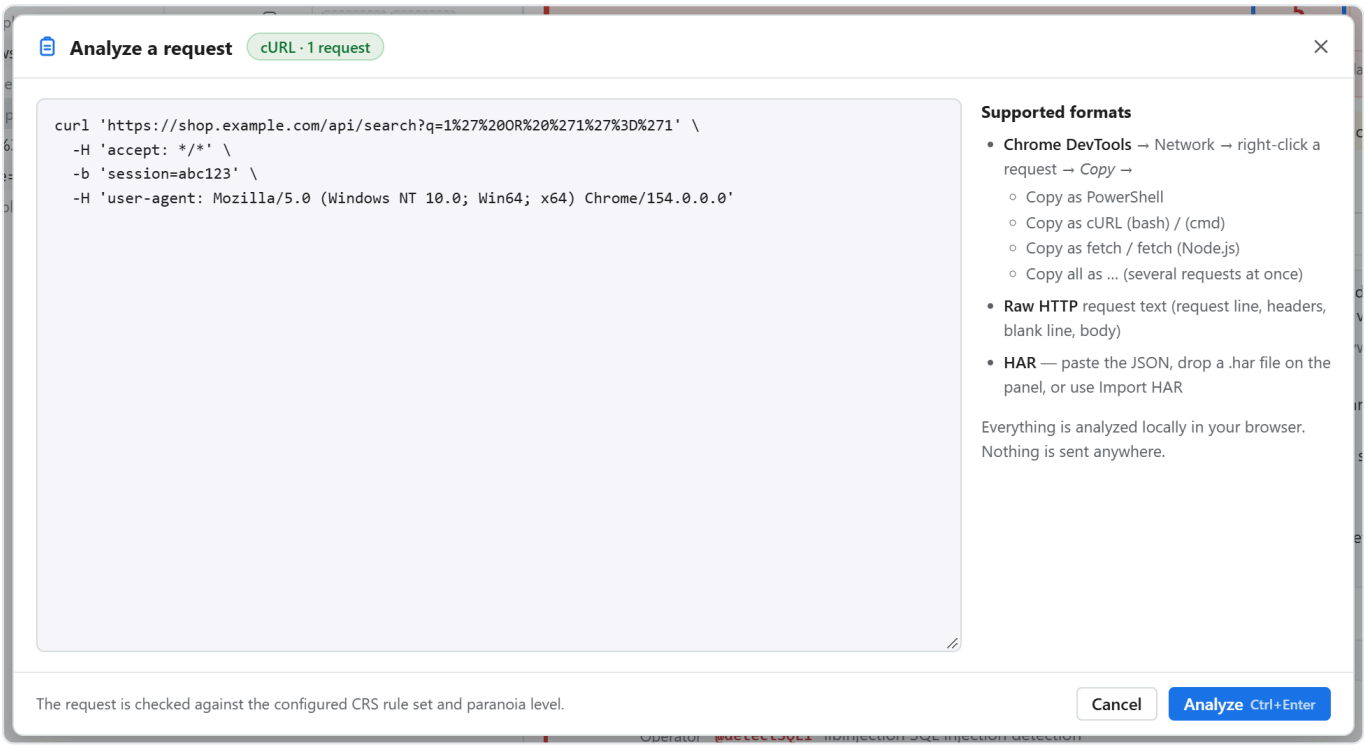
Useful for requests from colleagues, tickets or other browsers, or to test a modified request.

1 Copy the request in DevTools

Network tab → right-click the request → **Copy** → **Copy as PowerShell** (best: includes cookies and user agent), **Copy as cURL (bash) / (cmd)**, **Copy as fetch**, or **Copy all as …** for many requests.

2 Paste into the panel

Press `Ctrl` + `V` in the OWASP CRS tab or click **Paste request**. The format is detected automatically; click **Analyze** (`Ctrl` + `Enter`).

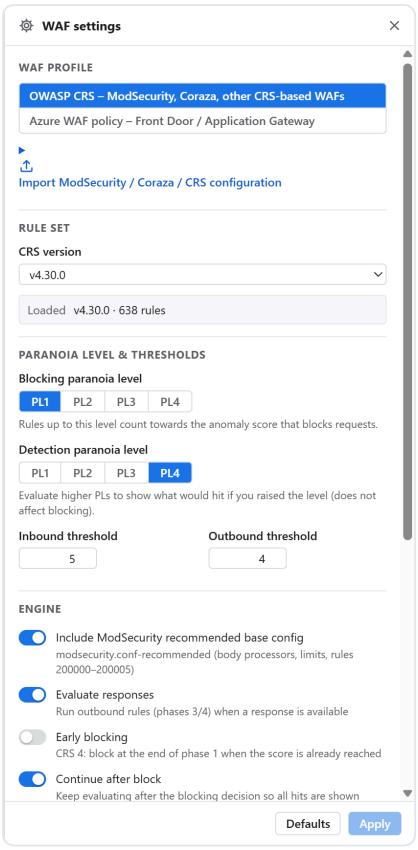


The paste dialog detects the format (here cURL, 1 request).

FORMAT	NOTES
PowerShell	**Recommended. HTTP/2 pseudo-headers (`authority`, `method`, `path`, `scheme`) are recognised; cookies from `$session.Cookies` are included.**
cURL (bash / cmd)	**Includes cookies (`-b`). Note: cURL copies omit the browser's `Accept-Encoding`.**
fetch / fetch (Node.js)	**Browser fetch copies contain no cookies and no User-Agent – may cause extra PL2 hits (920320 "missing User-Agent").**
Raw HTTP	**`POST /path HTTP/1.1`, headers, blank line, body. Separate several requests with a line `###`.**
HAR	**Import HAR or drop a `.har` file (Network tab → ⤓ Export HAR).**

Sensitive data. Copied requests contain session cookies and tokens. The extension keeps them in memory only and never sends them anywhere – but take care where else you paste them.

13 Settings



SETTING	MEANING
CRS version	**v4.30.0 (latest), v4.25.2 (LTS), v3.3.10 (closest to many managed WAFs based on CRS 3.x).**
Blocking paranoia level	**Same as the toolbar PL buttons: rules up to this level count.**
Detection paranoia level	**Rules up to this level are evaluated and shown, without affecting the verdict. Keep at PL4.**
Inbound / outbound threshold	**Score at which requests / responses are blocked (CRS defaults 5 / 4).**
ModSecurity base config	**JSON/XML body parsing, 128 KB body limit, invalid body → 400. Keep on.**
Evaluate responses	**Also checks HTML/text responses for data leakage (stack traces, SQL errors).**
Early blocking	**CRS 4 option: block already after the request headers.**
Continue after block	**Keep evaluating after the blocking decision so all hits are listed.**
HTTP version	**Simulate the protocol the WAF sees (e.g. HTTP/1.1 behind a CDN).**
Custom rules before / after CRS	**Your WAF's own configuration: e.g. `tx.allowed_methods` (CRS blocks PUT/PATCH/DELETE by default!), exclusions.**

> **Example – allow REST methods like the WAF does:**
> `SecAction "id:900200,phase:1,pass,nolog,setvar:'tx.allowed_methods=GET HEAD POST OPTIONS PUT PATCH DELETE'"`

14 FAQ & troubleshooting

QUESTION	ANSWER
The OWASP CRS tab is missing.	**Check the extension is enabled on `chrome://extensions`, then close and reopen DevTools. Look behind the » overflow in the tab bar.**
The list stays empty.	**Reload the page with DevTools open; check the record button is red. Requests made before DevTools was opened are not captured.**
Images and scripts are listed too.	**Yes – the WAF inspects every request (cookies and query strings of images can trigger rules). Use the type filters (*Fetch/XHR*, *Doc* …) to focus.**
Our WAF did not block a request the extension marks as blocked (or vice versa).	**See section 10: Detection mode, Front Door tier, policy association, disabled rules and DRS differences. Rule ids and the *Log data* line help to match WAF logs.**
DELETE / PUT / PATCH are blocked (911100).	**CRS only allows GET, HEAD, POST, OPTIONS by default. Most API deployments extend `tx.allowed_methods` – add the setting shown in section 13. On Azure, see sections 10–11.**
Large uploads / bodies are blocked (ENGINE 413).	**ModSecurity rejects bodies above 128 KB (without files) by default. Your WAF may use a different limit.**
“Rule 955100: data file not available”.	**Windows Defender removed a CRS signature file for web-shell detection in responses. Only that response rule is affected; request analysis is complete.**
Is any data sent to a server?	**No. Rules and analysis run entirely inside the extension.**

Glossary

CRS	**OWASP Core Rule Set – the generic attack-detection rules used by ModSecurity, Coraza and many cloud WAFs.**
Anomaly score	**Points accumulated by matching rules; compared with the threshold at the end of the request.**
Paranoia level (PL)	**How strict CRS is (1–4). Higher = more rules, more false positives.**
False positive	**A legitimate request that triggers a rule.**
Exclusion	**WAF configuration that stops a rule from inspecting a specific variable or endpoint.**
ARGS / REQUEST_COOKIES / REQUEST_HEADERS	**ModSecurity names for query/body parameters (JSON fields as `json.field`), cookies and headers.**